

BAGIAN B: HYBRID COMPUTING

Implementasi dan Evaluasi Kinerja Multi-Threading, Multi-Processing,
serta Message Passing Interface (MPI) pada Pemrosesan Data Hibrid

Disusun Oleh:

MUHAMMAD KENT FAIQ ARIFIEN

NPM: 247006111192 | Kelas G

Parameter Khusus: Digit Terakhir NPM = 2 (A = 2)

Dosen Pengampu:

Ir. Randi Rizal, S.T., M.Kom., Ph.D.

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK — UNIVERSITAS SILIWANGI

2026

RINGKASAN EKSEKUTIF

Laporan ini menyajikan analisis komprehensif implementasi komputasi hibrid (*hybrid computing*) yang memadukan paradigma *distributed-memory* (MPI) dan *shared-memory* (Multi-Threading dan Multi-Processing) pada arsitektur CPU 13th Gen Intel Core i5-13450HX (14 Physical Cores, 20 Logical Threads). Pengujian mencakup tiga modul utama praktikum: B1 (Hybrid Pipeline dengan Bounded Queue), B2 (Strong Scaling MPI + ProcessPool pada Monte Carlo Pi), dan B3 (Global Word Count teks nyata menggunakan ThreadPool vs ProcessPool).

Hasil eksperimen menunjukkan bahwa sinkronisasi antrean terbatas ($Q_MAX = 4$) menghasilkan efek *backpressure* yang menstabilkan konsumsi memori saat laju produsen melampaui konsumen. Pada komputasi Monte Carlo, penambahan worker mampu memberikan percepatan (*speedup*) hingga 2.21x, namun mengalami degradasi tajam (*oversubscription penalty*) saat alokasi proses (16 worker) melampaui batas core fisik mesin uji. Pada tugas pembacaan teks nyata B3, ThreadPoolExecutor terbukti 33.1x lebih cepat daripada ProcessPoolExecutor akibat pelepasan Python GIL saat operasi disk I/O dan minimnya overhead inisialisasi thread dibandingkan spawning proses di Windows.

BAB 1: SPESIFIKASI SISTEM DAN LINGKUNGAN PENGUJIAN

Seluruh eksperimen dijalankan secara empiris pada sistem laptop pengujian pribadi dengan spesifikasi perangkat keras dan perangkat lunak sebagai berikut:

Komponen Sistem	Spesifikasi Pengujian
Model Prosesor	13th Gen Intel® Core™ i5-13450HX
Arsitektur Core	14 Core Fisik (6 P-Cores + 8 E-Cores), 20 Logical Threads
Base / Turbo Clock	2.40 GHz (Base) / 4.60 GHz (Max Turbo)
Memori RAM	16 GB DDR5 4800 MT/s Dual-Channel
Penyimpanan	477 GB NVMe PCIe 4.0 SSD
Sistem Operasi	Windows 11 Pro 64-bit (x64)
Python Runtime	Python 3.11.9 (64-bit)
Implementasi MPI	Microsoft MPI (MS-MPI) v10.1.12498.52
Pustaka Python	mpi4py 4.1.2, NumPy 2.4.6, Pandas 3.0.6, Matplotlib 3.11.2

BAB 2: BAGIAN B1 — HYBRID PIPELINE

2.1 Arsitektur Pipeline dan Matriks Eksperimen

Sistem pipeline dirancang dengan arsitektur produsen-konsumen multi-tahap: (1) Thread pembaca (Loader Threads) memuat berkas teks secara asinkron; (2) Bounded Queue (kapasitas Q_MAX) menjadi penyangga sinkronisasi thread-safe; dan (3) Worker multi-proses memproses parsing teks secara paralel murni tanpa intervensi GIL. Dengan parameter mahasiswa A = 2, total berkas teks sintetis adalah 120 berkas. Matriks pengujian menguji 24 kombinasi lengkap: Loader Threads in {1, 2, 4}, Workers in {1, 2, 4, 14}, dan Q_MAX in {4, 32}.

2.3 Visualisasi Grafik Hasil Eksperimen B1

2.4 Pembahasan Kritis dan Analisis Backpressure

1. Titik Optimal: Konfigurasi paling unggul diraih pada Eksperimen #22 (4 Loaders, 4 Workers, Q_MAX=32) dengan throughput 1029.05 file/detik dan waktu 0.1166 detik.

2. Pola Skalabilitas & Penurunan pada 14 Worker: Throughput naik stabil dari 1 worker (653.09 f/s) menuju 4 worker (927.52 f/s). Namun saat dipaksa mencapai 14 worker, throughput anjlok ke 519.65 f/s. Hal ini mencerminkan tingginya biaya overhead konkurensi (lock contention pada queue IPC dan context switching OS) yang melebihi beban pemrosesan 120 berkas teks sintetis.

3. Evaluasi Backpressure: Pada Q_MAX = 4, buffer lekas terisi penuh saat loader membaca disk dengan cepat. Pemanggilan queue.put() memblokir thread loader secara otomatis, menahan produsen dan mencegah memory overflow. Sebaliknya, pada Q_MAX = 32 buffer lebih longgar sehingga throughput sedikit meningkat (+4.2%), tetapi rata-rata latency melonjak 3.9x lipat akibat waktu antrre (*residence time*) berkas yang lebih panjang.

BAB 3: BAGIAN B2 — STRONG SCALING MPI + PROCESSPOOL

3.1 Arsitektur Hibrid dan Koreksi Bug Slide 20

Sistem B2 mengombinasikan MPI (mpi4py) untuk alokasi domain antar-proses dengan ProcessPoolExecutor untuk pembagian thread/proses multi-core lokal dalam menghitung estimasi Monte Carlo Pi. Parameter beban kerja diatur sebesar SAMPLES_PER_TASK = 220.000 sampel per task. Koreksi fatal terhadap kode slide 20 dilakukan dengan menambahkan penutup blok `if __name__ == '__main__':`, mengisolasi pembuatan pool lokal dari communicator MPI, serta menyinkronkan hasil melalui `comm.reduce(op=MPI.SUM)`.

3.3 Visualisasi Grafik Hasil Eksperimen B2

3.4 Analisis Speedup dan Fenomena Oversubscription

1. Akselerasi Skalabilitas Optimal: Dari baseline waktu T1 = 0.3889 detik, peningkatan performa tertinggi dicapai oleh kombinasi 2 ranks x 2 workers (0.1763 detik, Speedup 2.2059x) dan 4 ranks x 1 worker (0.1765 detik, Speedup 2.2034x). Hal ini menunjukkan distribusi beban merata dengan overhead komunikasi MPI minimal.

2. Terjadinya Oversubscription pada 4 Ranks x 4 Workers: Pada konfigurasi 4 ranks x 4 workers, sistem mengalokasikan 4 x 4 = 16 sub-proses worker di samping 4 proses utama MPI rank (total 20 proses aktif). Jumlah ini melampaui 14 Physical Cores CPU laptop pengujian.

3. Analisis Dampak Oversubscription: Waktu komputasi melonjak drastis menjadi 0.2748 detik, speedup runtuh ke 1.4152x, dan efisiensi jatuh ke angka 0.0885 (8.85%). Penurunan tajam ini adalah bukti nyata fenomena *CPU context-switching thrashing*, di mana CPU menghabiskan porsi siklus clock yang signifikan untuk menyimpan dan memuat register proses ketimbang melakukan komputasi floating-point nyata.

BAB 4: BAGIAN B3 — GLOBAL WORD COUNT (THREADPOOL VS PROCESSPOOL)

4.1 Arsitektur Pengujian pada Dataset Teks Nyata

Bagian B3 menguji pemrosesan kata pada 32 berkas artikel ilmiah teks nyata (bertema komputasi paralel dan sistem terdistribusi). Dataset didistribusikan secara berimbang antar-rank MPI, kemudian setiap rank mengeksekusi perhitungan kata lokal menggunakan dua metode alternatif: ThreadPoolExecutor dan ProcessPoolExecutor.

4.3 Visualisasi Grafik Hasil Eksperimen B3

4.4 Analisis Kritis: Mengapa ThreadPool 33.1x Lebih Cepat?

Hasil pengujian empiris mengonfirmasi keunggulan mutlak ThreadPoolExecutor (rata-rata total waktu 0.0046s) dibandingkan ProcessPoolExecutor (rata-rata 0.1523s), atau sekitar 33.1x lebih cepat. Penjelasanannya adalah:

1. Karakteristik I/O-Bound & Pelepasan GIL: Tahap membaca 32 berkas dari SSD merupakan operasi I/O-bound. Interpreter CPython secara otomatis melepaskan Global Interpreter Lock (GIL) ketika thread memanggil fungsi I/O sistem, sehingga multi-threading berjalan secara konkuren tanpa terhambat GIL.

2. Overhead Spawning Proses di Windows: Di sistem Windows (yang tidak memiliki mekanisme `fork()`), ProcessPoolExecutor harus menggunakan model `spawn` yang mengalokasikan ruang memori terpisah, memuat interpreter Python baru, dan menserialisasi data via IPC. Biaya inisialisasi ini memakan waktu statis 0.10 - 0.25 detik.

3. Beban Komputasi Regex yang Sangat Singkat: Waktu komputasi murni tokenisasi kata (Processing Time) hanya memakan waktu ~0.0009 detik. Akibatnya, pada ProcessPoolExecutor, 99% waktu eksekusi terbuang hanya untuk proses inisialisasi dan komunikasi IPC, bukan untuk komputasi sesungguhnya.

BAB 5: KESIMPULAN DAN REKOMENDASI ARSITEKTUR

Berdasarkan keseluruhan hasil perancangan, eksperimen empiris, dan analisis komputasi hibrid, ditarik kesimpulan dan panduan arsitektural sebagai berikut:

- 1. Mekanisme Backpressure:** Bounded queue adalah komponen vital dalam pipeline produsen-konsumen untuk mengontrol laju data dan mencegah lonjakan memori tak terbatas.
- 2. Batas Skalabilitas Fisik:** Mengalokasikan proses worker yang melebihi jumlah core fisik prosesor (oversubscription) menimbulkan penalti performa yang parah. Konfigurasi komputasi paralel harus diselaraskan dengan batas core fisik sistem.
- 3. Pemilihan Paradigma:** Untuk tugas I/O-bound dengan beban komputasi ringan hingga sedang, ThreadPoolExecutor merupakan pilihan paling efisien karena overhead inisialisasi mendekati nol. ProcessPoolExecutor hanya tepat digunakan pada komputasi CPU-bound berat di mana durasi komputasi jauh melampaui biaya spawning proses.